



Кейс 1. Генерация CRUD для системы заявок

Команда разрабатывает внутреннюю систему технической поддержки. Сотрудники компании смогут создавать заявки, связанные с неисправностями оборудования и программного обеспечения. На текущем этапе проекта уже существует серверное приложение и подключена база данных, однако модуль работы с заявками ещё не реализован.

В базе данных должна храниться сущность `Ticket`. Для каждой заявки необходимо хранить идентификатор, заголовок, описание проблемы, статус, дату создания и идентификатор сотрудника, создавшего заявку.

Статус заявки может принимать значения `new`, `in_progress`, `resolved` и `closed`.

Для сущности требуется реализовать стандартные CRUD-операции. Система должна позволять создать новую заявку, получить список заявок, получить одну заявку по идентификатору, изменить существующую заявку и удалить её.

Проект использует Python, FastAPI, SQLAlchemy и PostgreSQL.

Существующая модель данных представлена в упрощённом виде:

```
class Ticket:
    id: int
    title: str
    description: str
    status: str
    created_at: datetime
    user_id: int
```

При создании заявки клиент передаёт заголовок, описание проблемы и идентификатор пользователя. Новая заявка должна автоматически получать статус `new`.

При получении списка заявок необходимо предусмотреть возможность фильтрации по статусу.

При запросе заявки по несуществующему идентификатору API должен возвращать сообщение о том, что объект не найден.

При изменении заявки пользователь может изменить заголовок, описание и статус. Поля `id`, `created_at` и `user_id` изменять через данный запрос нельзя.

Команда рассматривает использование ИИ для подготовки первоначальной реализации нового CRUD-модуля.

Рабочая задача заключается в создании модуля работы с заявками, соответствующего существующему технологическому стеку и указанным требованиям.

Кейс 2. Миграция части проекта на новый стек

Компания поддерживает внутренний сервис управления товарами. Проект существует несколько лет и был создан на Flask.

Команда постепенно переводит серверную часть приложения на FastAPI. Полностью переписывать проект за один этап нельзя, поэтому миграция выполняется по отдельным модулям.

Необходимо перенести модуль получения информации о товарах.

Старый Flask-маршрут выглядит следующим образом:

```
@app.route("/products/<int:product_id>")
def get_product(product_id):
    product = product_service.find(product_id)

    if product is None:
        return {"error": "Product not found"}, 404

    return product.to_dict()
```

В новой части проекта уже используется FastAPI.

Пример структуры нового приложения:

```
app/
  main.py
  routers/
  services/
  models/
  schemas/
```

Обращение к базе данных должно оставаться внутри сервисного слоя. Маршрут не должен самостоятельно выполнять SQL-запросы.

В новом проекте для входных и выходных данных используются Pydantic-модели.

Существующий сервис `product_service.find(product_id)` пока должен быть сохранён. Его переписывание не входит в текущую задачу.

При отсутствии товара клиент должен получать HTTP-статус `404`.

URL нового API должен остаться совместимым с существующими клиентскими приложениями:

```
GET /products/{product_id}
```

В проекте уже существуют мобильное приложение и административная панель, использующие данный адрес. Изменение формата ответа может потребовать изменений в этих клиентах.

Команда рассматривает применение ИИ для выполнения части миграции и адаптации существующего кода под новый стек.

Рабочая задача заключается в переносе указанной функциональности с Flask на FastAPI без нарушения заданных ограничений.

Кейс 3. Автодополнение при разработке функции

Разработчик работает над модулем интернет-магазина. Необходимо реализовать функцию расчёта итоговой стоимости заказа.

В систему передаётся список товаров. Каждый товар содержит цену и количество.

Фрагмент исходных данных может выглядеть следующим образом:

```
items = [{"price": 1200, "quantity": 2}, {"price": 500, "quantity": 1}]
```

Разработчик начинает писать функцию:

```
def calculate_total(items):  
    total = 0  
  
    for item in items:
```

Среда разработки подключена к системе интеллектуального автодополнения кода. После ввода очередной строки система автоматически предлагает продолжение функции.

Функция должна вычислять общую стоимость всех позиций заказа.

Если список товаров пустой, функция должна возвращать `0`.

Цена товара хранится в поле `price`, количество — в поле `quantity`.

В дальнейшем в функцию потребуется добавить обработку скидок, однако в текущей версии скидки учитывать не требуется.

После написания нескольких строк разработчик получает от системы автодополнения один или несколько вариантов продолжения кода.

Предложения могут содержать как отдельную строку, так и сразу несколько строк реализации.

Разработчик может принять предложение полностью, принять только его часть, отклонить предложение и продолжить написание функции самостоятельно либо изменить предложенный вариант после вставки.

В команде планируется сравнить обычное ручное написание небольших функций и разработку с использованием интеллектуального автодополнения.

Рабочая задача заключается в завершении функции `calculate_total(items)` в соответствии с указанными требованиями с возможностью использования автоматически предлагаемых фрагментов кода.

Кейс 4. Авторевью изменений перед слиянием

Команда разрабатывает API интернет-магазина. Один из разработчиков подготовил изменение функции, отвечающей за получение пользователя по идентификатору.

Изменение отправлено в Pull Request перед объединением с основной веткой проекта.

В Pull Request появился следующий фрагмент:

```
def get_user(user_id):
    user = database.get_user(user_id)

    if user:
        return user

    return None
```

В этом же Pull Request другой метод был изменён следующим образом:

```
def delete_user(user_id):
    database.delete_user(user_id)
    return {"status": "deleted"}
```

В проекте действует правило: операции получения, изменения и удаления объектов должны корректно обрабатывать ситуацию, когда объект с указанным идентификатором отсутствует.

Для API также важно, чтобы внешнее поведение методов оставалось предсказуемым для клиентских приложений.

Pull Request содержит около 350 изменённых строк в восьми файлах. Помимо показанных функций, были изменены модели, несколько API-маршрутов и тесты.

В проекте уже настроено автоматическое выполнение тестов после создания Pull Request. На момент начала ревью основная часть тестов проходит успешно.

Команда рассматривает возможность добавить автоматический анализ Pull Request с использованием ИИ до начала проверки человеком.

Система авторевью получает изменённые фрагменты файлов и формирует комментарии к потенциально проблемным местам.

Результат автоматического анализа не изменяет код самостоятельно. Он добавляет замечания непосредственно в Pull Request, после чего разработчик или ревьюер решает, необходимо ли вносить изменения.

В рамках проекта особенно важны корректность обработки ошибок, сохранение совместимости API, читаемость кода и отсутствие очевидных дефектов в изменённых участках.

Рабочая задача заключается в проведении ревью представленных изменений и остальных изменений Pull Request с использованием доступного механизма автоматического анализа кода.